

CODES:

BankModules.v:

```
1  /*
2  * Yigit Suoglu
3  * Bank Queue Management System
4  * Designed to work on Digilent Basys 3 but it should work on any FPGA
5  * Size of password switches and leds output can be changed without effecting functionality
6  * Licence: CERN-OHL-W
7  */
8  `timescale 1ns / 1ps
9
10 module bankSystem(ssd0, ssd1, ssd2, ssd3, passSW, Teller1btn, Teller0btn,
11   bankLed, Teller1led, Teller0led, rst, clk, customerBTN, counterSW, bankSW, chPASS, Wait);
12
13
14   input clk, rst; //clock and reset
15   output [4:0] ssd1, ssd2, ssd3, ssd0; //what each ssd should be in abcdefg format
16   input [11:0] passSW; //password switches
17   input Teller1btn, Teller0btn; //teller buttons
18   input customerBTN; //button for customer to take number
19   input counterSW; //open-close counter
20   input bankSW; //open-close bank
21   input chPASS; //change password
22   output reg bankLed, Teller1led, Teller0led; //bank & counter open leds
23   reg [7:0] callingCus, totalCus, teller1call, teller0call;
24   //output [12:0] leds; //other leds to show stage
25   reg [11:0] passT1, passT0; //holds passwords for tellers default 12'd0;
26   reg trigQueNumTake; //trigger to show taken Queue number
27   output reg Wait; //waiting something, works with operation signal
28   reg waitswho; //shows system waiting who, work with wait signal
29
30   wire blinkOff, queueActive;
31   wire [7:0] waitingCus; //remaning customer
32   wire readyForClose; //no customer left and both counters are closed
33   wire [2:0] operation; //combined operation switches
34   wire notRdyCls;
35
36   assign notRdyCls = ~readyForClose & bankSW;
37   assign operation = {bankSW, counterSW, chPASS}; //order: bank, counter, change password
38   assign readyForClose = ~(Teller1led | Teller0led) & (callingCus == totalCus) | ~bankLed;
39   assign waitingCus = totalCus - callingCus;
40
41   always@(posedge clk or posedge rst) //Calling customer
42   begin
43     if(rst)
44     begin
45       teller1call <= 8'b0;
46       teller0call <= 8'b0;
47       callingCus <= 8'b0;
48     end
49   else if((operation == 3'b0) & (waitingCus != 8'b0))
50   begin
51     /* if(Teller0btn & Teller1btn & Teller1led & Teller0led) //both tellers call at same time cycle, not necessary
52     begin
53       teller1call <= callingCus + 8'b10;
54       teller0call <= callingCus + 8'b1;
55       callingCus <= callingCus + 8'b10;
56     end
57     else if(Teller0btn & Teller0led)
58     begin
59       teller0call <= callingCus + 8'b1;
60       callingCus <= callingCus + 8'b1;
61     end
62     else if(Teller1btn & Teller1led)
63     begin
64       teller1call <= callingCus + 8'b1;
65       callingCus <= callingCus + 8'b1;
66     end
67   end
68   end
69   always@(posedge clk or posedge rst) //operations
70   begin
71     if(rst)
72     begin
73       bankLed <= 0;
74       Teller0led <= 0;
75       Teller1led <= 0;
76       Wait <= 0;
77       passT0 <= 12'd0;
78       passT1 <= 12'd0;
79     end
80   else
81   case (operation)
82     3'b100: //bank open-close
83     begin // #1
84       if(readyForClose)
85       begin // #2
86         if (Wait) //if one of the tellers entered pasword
87         case(waitswho)
88           0: if((passSW == passT0) & Teller0btn)
89           begin // #3
90             bankLed <= ~bankLed;
91             Wait <= 0;
92           end // #3
93           1: if((passSW == passT1) & Teller1btn)
94           begin // #3
95             bankLed <= ~bankLed;
96           end // #3
97         end case
98       end
99     end
100   end case
```

```

97         wait <= 0;
98     end // #3
99 endcase
100 else
101     begin // #3
102         if (passSW == passT0) & Teller0btn)
103             begin // #4
104                 wait <= 1;
105                 waitswwho <= 1;
106             end // #4
107         else if (passSW == passT1) & Teller1btn)
108             begin // #4
109                 wait <= 1;
110                 waitswwho <= 0;
111             end // #4
112         end // #3
113     end // #2
114 end // #1
115 3'b010: //counter open-close
116 begin
117     if (passSW == passT0) & Teller0btn)
118         Teller0led <= ~Teller0led;
119     else if (passSW == passT1) & Teller1btn)
120         Teller1led <= ~Teller1led;
121     end
122 3'b001: //change password
123 begin
124     if (wait) //if one of the tellers entered password
125         case (waitswwho)
126             0: if (Teller0btn)
127                 passT0 <= passSW;
128             1: if (Teller1btn)
129                 passT1 <= passSW;
130         endcase
131     else
132         begin // #3
133             if (passSW == passT0) & Teller0btn)
134                 begin // #4
135                     wait <= 1;
136                     waitswwho <= 0;
137                 end // #4
138             else if (passSW == passT1) & Teller1btn)
139                 begin // #4
140                     wait <= 1;
141                     waitswwho <= 1;
142                 end // #4
143             end // #3
144         end

```

```

145     default: wait <= 0;
146 endcase
147 end
148
149 always @(posedge clk or posedge rst) //Queue number taking
150 begin
151     if (rst)
152         begin
153             trigQueueNumTake <= 0;
154             totalCus <= 8'd0;
155         end
156     else if (bankLed)
157         begin
158             if (customerBTN)
159                 begin
160                     trigQueueNumTake <= 1;
161                     totalCus <= totalCus + 8'b1;
162                 end
163             else
164                 begin
165                     trigQueueNumTake <= 0;
166                 end
167             end
168         else
169             begin
170                 totalCus <= 8'd0;
171                 trigQueueNumTake <= 0;
172             end
173         end
174
175 displaySelection sssSelector(.dig0(ssd0), .dig1(ssd1), .dig2(ssd2), .dig3(ssd3),
176     .bankClosed(bankLed), .queueNum(queueActive), .totalCus(totalCus),
177     .teller1call(teller1call), .teller0call(teller0call),
178     .notRdyCIs(notRdyCIs), .waitingCus(waitingCus), .blinkOff(blinkOff));
179
180 queueNumTakeProces queueMng(.trigQueueNumTake(trigQueueNumTake), .clk(clk), .rst(rst),
181     .queueActive(queueActive), .blinkOff(blinkOff));
182
183 endmodule
184
185 module displaySelection(dig0, dig1, dig2, dig3, bankClosed, queueNum, totalCus,
186     teller1call, teller0call, notRdyCIs, waitingCus, blinkOff /*, pswrdCH*/);
187     parameter a = 5'd10, b = 5'd11, c = 5'd12, d = 5'd13, h = 5'd19;
188     parameter e = 5'd14, f = 5'd15, o = 5'd16, L = 5'd17, p = 5'd18, r = 5'd20;
189
190     output reg [4:0] dig0, dig1, dig2, dig3;
191     input bankClosed, notRdyCIs;
192     input blinkOff, queueNum /*, pswrdCH*/;

```

```

193 input [7:0] totalCus, teller1call, teller0call, waitingCus;
194
195 always@(*)
196   if(!bankClosed) //bank closed
197     begin
198       dig3 = c;
199       dig2 = L;
200       dig1 = 5'd5; //s
201       dig0 = d;
202     end
203   else if(blinkOff) //ssd's are not illuminated (used for blinking)
204     begin
205       dig3 = 5'b11111;
206       dig2 = 5'b11111;
207       dig1 = 5'b11111;
208       dig0 = 5'b11111;
209     end
210   else if(queueNum) //show taken number
211     begin
212       dig3 = 5'b11111;
213       dig2 = 5'b11111;
214       dig1 = {1'b0, totalCus[7:4]};
215       dig0 = {1'b0, totalCus[3:0]};
216     end
217   else if(notRdyCls) //show taken number
218     begin
219       dig3 = r;
220       dig2 = 5'd12; //c
221       dig1 = {1'b0, waitingCus[7:4]};
222       dig0 = {1'b0, waitingCus[3:0]};
223     end
224   /* else if(pswrdCH) //ch pasword mode
225     begin
226       dig3 = 5'd12; //c
227       dig2 = h;
228       dig1 = p; //
229       dig0 = 5'd5; //s
230     end */
231   else
232     begin //last called number
233       dig3 = {1'b0, teller1call[7:4]};
234       dig2 = {1'b0, teller1call[3:0]};
235       dig1 = {1'b0, teller0call[7:4]};
236       dig0 = {1'b0, teller0call[3:0]};
237     end
238 endmodule
239
240 module queueNumTakeProces(trigQueueNumTake, clk, rst, queueActive, blinkOff);
241 input trigQueueNumTake, clk, rst;
242 output reg queueActive;
243 output blinkOff;
244 reg [1:0] state;
245 wire oneSecSig, quarSecSig, quaTrig, quaTrigRep;
246 reg quarSecSig_s;
247
248 quarSecWait quaCount(.start(quaTrig), .rst(rst), .clk(clk), .lastCycle(quarSecSig));
249 oneSecWait oneCount(.start(trigQueueNumTake), .rst(rst), .clk(clk), .lastCycle(oneSecSig));
250
251 assign quaTrig = trigQueueNumTake | quaTrigRep;
252 assign blinkOff = state[0];
253
254 always@(posedge clk or posedge rst) //state transaction
255   begin
256     if (rst)
257       begin
258         state <= 2'b0;
259       end
260     else
261       case (state)
262         2'b11: state <= 2'b0;
263         default: state <= state + {1'b0, quarSecSig};
264       endcase
265   end
266
267 always@(posedge clk or posedge rst) //queueActive signal
268   if (rst)
269     queueActive <= 0;
270   else if (oneSecSig)
271     queueActive <= 0;
272   else if (trigQueueNumTake)
273     queueActive <= 1;
274
275   assign quaTrigRep = quarSecSig_s & (state != 2'd3);
276
277   always@(posedge clk)
278     begin
279       quarSecSig_s <= quarSecSig;
280     end
281
282
283
284 endmodule

```

Counters.v:

```
1 //Yigit Suoglu
2 //This file contains counters to count 1 sec, 0.5 sec and 0.25 sec.
3 //Note: parameters should be updated according to device clock
4 //Parameters are for clock frequency of 100MHz (period of 10 ns)
5 //Licence: CERN-OHL-W
6 timescale 1ns / 1ps
7
8 module oneSecWait(start, rst, clk, lastCycle);
9     parameter oneSec = 27'd100000000; //frequency of device clock, use 4'b1000 for sim
10     parameter counterSize = 27; //size of reg, use 4 for sim
11
12     input start, rst, clk;
13     output lastCycle;
14
15     reg [(counterSize - 1):0] count;
16
17     assign lastCycle = (count == oneSec);
18
19     always@(posedge clk or posedge rst)
20     begin
21         if(rst)
22             count <= 0;
23         else if(count == oneSec)
24             count <= 0;
25         else if(start)
26             count <= 1;
27         else if(count != 0)
28             count <= count + 1;
29         else
30             count <= count;
31     end
32
33 endmodule //1 sec
34
35 module halfSecWait(start, rst, clk, lastCycle);
36     parameter oneSec = (27'd100000000 >> 1); //frequency of device clock, use 3'b100 for sim
37     parameter counterSize = (27 - 1); //size of reg, use 3 for sim
38
39     input start, rst, clk;
40     output lastCycle;
41
42     reg [(counterSize - 1):0] count;
43
44     assign lastCycle = (count == oneSec);
45
46     always@(posedge clk or posedge rst)
47     begin
48         if(rst)
49             count <= 0;
50         else if(count == oneSec)
51             count <= 0;
52         else if(start)
53             count <= 1;
54         else if(count != 0)
55             count <= count + 1;
56         else
57             count <= count;
58     end
59
60 endmodule //0.5 sec
61
62 module quarSecWait(start, rst, clk, lastCycle);
63     parameter oneSec = (27'd100000000 >> 2); //frequency of device clock, use 2'b10 for sim
64     parameter counterSize = (27 - 2); //size of reg, use 2 for sim
65
66     input start, rst, clk;
67     output lastCycle;
68
69     reg [(counterSize - 1):0] count;
70
71     always@(posedge clk or posedge rst)
72     begin
73         if(rst)
74             count <= 0;
75         else if(count == oneSec)
76             count <= 0;
77         else if(start)
78             count <= 1;
79         else if(count != 0)
80             count <= count + 1;
81         else
82             count <= count;
83     end
84
85 assign lastCycle = (count == oneSec);
86
87 endmodule //0.25 sec
```

SevenSegmentDisplayDecoders.v:

```

1  /*
2  * Yigit Suoglu
3  * Licence: CERN-OHL-W
4  * Out signals in form of abcdefg corresponding:
5  *   a
6  *   f b      Note: abcdefg signals should driven low to illuminate
7  *   g        corresponding segment.
8  *   e c
9  *   d
10 */
11 `timescale 1ns / 1ps
12
13 module int4bitToHexSSD(in, out);
14     parameter zero = 7'b0000001, one = 7'b1001111, two = 7'b0010010;
15     parameter thr = 7'b0000110, four = 7'b1001100, five = 7'b0100100;
16     parameter six = 7'b0100000, svn = 7'b0001111, eght = 7'b0000000;
17     parameter nine = 7'b0000100, A = 7'b0001000, B = 7'b1100000;
18     parameter C = 7'b0110001, D = 7'b1000010, E = 7'b0110000, F = 7'b0111000;
19
20     input [3:0] in;
21     output reg [6:0] out;
22
23     always@(*)
24     case (in)
25         4'b0: out = zero;
26         4'b1: out = one;
27         4'b10: out = two;
28         4'b11: out = thr;
29         4'b100: out = four;
30         4'b101: out = five;
31         4'b110: out = six;
32         4'b111: out = svn;
33         4'b1000: out = eght;
34         4'b1001: out = nine;
35         4'b1010: out = A;
36         4'b1011: out = B;
37         4'b1100: out = C;
38         4'b1101: out = D;
39         4'b1110: out = E;
40         4'b1111: out = F;
41     endcase
42 endmodule
43
44 module int5bitToHexSSD_bankVer(in, out);
45     parameter zero = 7'b0000001, one = 7'b1001111, two = 7'b0010010; //in = 0, 1, 2
46     parameter thr = 7'b0000110, four = 7'b1001100, five = 7'b0100100; //in = 3, 4, 5 ~ 5 = s in ssd
47     parameter six = 7'b0100000, svn = 7'b0001111, eght = 7'b0000000; //in = 6, 7, 8
48     parameter nine = 7'b0000100, A = 7'b0001000, B = 7'b1100000; //in = 9, 10, 11
49     parameter C = 7'b0110001, D = 7'b1000010, E = 7'b0110000, F = 7'b0111000; //in = 12, 13, 14, 15
50     parameter o = 7'b1100010, L = 7'b1110001, empty = 7'b1111111; //in = 16, 17, others
51     parameter p = 7'b0011000, h = 7'b1101000, r = 7'b1111010; //in = 18, 19, 20
52
53     input [4:0] in;
54     output reg [6:0] out;
55
56     always@(*)
57     case (in)
58         5'b0: out = zero;
59         5'b1: out = one;
60         5'b10: out = two;
61         5'b11: out = thr;
62         5'b100: out = four;
63         5'b101: out = five;
64         5'b110: out = six;
65         5'b111: out = svn;
66         5'b1000: out = eght;
67         5'b1001: out = nine;
68         5'b1010: out = A;
69         5'b1011: out = B;
70         5'b1100: out = C;
71         5'b1101: out = D;
72         5'b1110: out = E;
73         5'b1111: out = F;
74         5'b10000: out = o;
75         5'b10001: out = L;
76         5'b10010: out = p;
77         5'b10011: out = h;
78         5'b10100: out = r;
79         default: out = empty;
80     endcase
81 endmodule

```

board.v:

```
1 `timescale 1ns / 1ps
2
3 module board(
4     input clk,
5     input rst,
6     input [11:0] passSW,
7     input T1btn,
8     input T0btn,
9     input Cbtn,
10    output bnkLED,
11    output t1LED,
12    output t0LED,
13    input counterSW,
14    input bankSW,
15    input chPASS,
16    output a,
17    output b,
18    output c,
19    output d,
20    output e,
21    output f,
22    output g,
23    output an0,
24    output an1,
25    output an2,
26    output an3,
27    output Wait
28 );
29
30 wire [4:0] ssd0_pre, ssd1_pre, ssd2_pre, ssd3_pre;
31 wire [6:0] ssd0, ssd1, ssd3, ssd2;
32 wire T1btn_c, T0btn_c, Cbtn_c;
33
34 bankSystem core(.ssd0(ssd0_pre), .ssd1(ssd1_pre), .ssd2(ssd2_pre), .ssd3(ssd3_pre), .passSW(passSW),
35 .Teller1btn(T1btn_c), .Teller0btn(T0btn_c), .bankLed(bnkLED), .Teller1led(t1LED), .Teller0led(t0LED),
36 .rst(rst), .clk(clk), .customerBTN(Cbtn_c), .counterSW(counterSW), .bankSW(bankSW), .chPASS(chPASS), .Wait(Wait));
37
38 int5bitToHexSSD_bankVer ssd0M(.in(ssd0_pre), .out(ssd0));
39 int5bitToHexSSD_bankVer ssd1M(.in(ssd1_pre), .out(ssd1));
40 int5bitToHexSSD_bankVer ssd2M(.in(ssd2_pre), .out(ssd2));
41 int5bitToHexSSD_bankVer ssd3M(.in(ssd3_pre), .out(ssd3));
42
43 ssd_ssdControl(.clk(clk), .rst(rst), .a0(ssd0[0]), .a1(ssd1[0]), .a2(ssd2[0]), .a3(ssd3[0]),
44 .b0(ssd0[1]), .b1(ssd1[1]), .b2(ssd2[1]), .b3(ssd3[1]), .c0(ssd0[2]), .c1(ssd1[2]), .c2(ssd2[2]), .c3(ssd3[2]),
45 .d0(ssd0[3]), .d1(ssd1[3]), .d2(ssd2[3]), .d3(ssd3[3]), .e0(ssd0[4]), .e1(ssd1[4]), .e2(ssd2[4]), .e3(ssd3[4]),
46 .f0(ssd0[5]), .f1(ssd1[5]), .f2(ssd2[5]), .f3(ssd3[5]), .g0(ssd0[6]), .g1(ssd1[6]), .g2(ssd2[6]), .g3(ssd3[6]),
47 .a(a), .b(b), .c(c), .d(d), .e(e), .f(f), .g(g), .an0(an0), .an1(an1), .an2(an2), .an3(an3));
48
49 debouncer t0BUT(.clk(clk), .rst(rst), .noisy_in(T0btn), .clean_out(T0btn_c));
50 debouncer t1BUT(.clk(clk), .rst(rst), .noisy_in(T1btn), .clean_out(T1btn_c));
51 debouncer cBUT(.clk(clk), .rst(rst), .noisy_in(Cbtn), .clean_out(Cbtn_c));
52
53 endmodule
```

debouncer.v:

```

1  timescale 1ns / 1ps
2
3  module debouncer(
4      input      clk,
5      input      rst,
6      input      noisy_in, // port from the push button
7      output reg  clean_out // port into the circuit
8  );
9
10
11  reg noisy_in_reg;
12  reg clean_out_tmp1; // will be used to detect rising edge
13  reg clean_out_tmp2; // will be used to detect rising edge
14
15  always@(posedge clk or posedge rst)
16  begin
17      if (rst)
18          begin
19              noisy_in_reg <= 0;
20              clean_out_tmp1 <= 0;
21              clean_out_tmp2 <= 0;
22              clean_out <= 0;
23          end
24      else
25          begin
26              noisy_in_reg <= noisy_in; // store the input
27              clean_out_tmp1 <= noisy_in_reg;
28
29              // rising edge detect
30              clean_out_tmp2 <= clean_out_tmp1;
31              clean_out <= ~clean_out_tmp2 & clean_out_tmp1; // it produce a single pulse during a risingedge
32          end
33      end
34  end
35
36  endmodule

```

ssd.v:

```

1  timescale 1ns / 1ps
2
3  module ssd( clk, rst, a0,a1,a2,a3,b0,b1,b2,b3,c0,c1,c2,c3,d0,d1,d2,d3,e0,e1,e2,e3,
4      f0,f1,f2,f3,g0,g1,g2,g3,a,b,c,d,e,f,g,an0,an1,an2,an3
5  );
6
7  input clk, rst; // set clock and reset as input(1 bit)
8  input a0,a1,a2,a3,b0,b1,b2,b3,c0,c1,c2,c3,d0,d1,d2,d3,e0,e1,e2,e3,f0,f1,f2,f3,g0,g1,g2,g3; //set our display data as input(1 bit)
9  output reg a,b,c,d,e,f,g,an0,an1,an2,an3; //set outputs also as registers (1 bit)
10
11  reg [2:0] state; //holds state number (3 bit)
12  reg [14:0] counter; //counter to slow the input clock(15 bit)
13
14  //in this always block the speed of the clock reduced by 25000 times so that display works properly
15  //for a clock frequency is different than 100MHz use a counter value that gives reduced clock frequency if ~4kHz.
16  //Line 12 and 23 should be changed accordingly)
17  always @ (posedge clk) begin //state counter
18      if(rst) begin //synchronous reset
19          state <= 0; //if reset set state and counter to zero
20          counter <= 0;
21      end else begin //else the counter untill 25000 (limit)
22          if(counter == 15'h61A8) //if equal to 25000 (limit)
23              begin
24                  if (state == 3'b100) //if it is in the last state return to state 1
25                      state <= 1;
26                  else
27                      state <= state + 1; //else go one state up
28              end
29          counter <= 0; //0 the counter because after it reaches 25000
30      end
31      else
32          counter <= counter + 1; //if not 25000 add 1
33      end
34  end
35
36
37
38  always@(posedge clk)
39  begin
40      if(rst) // if reset initialize the outputs
41          begin
42              an0 <= 1;
43              an1 <= 1;
44              an2 <= 1;
45              an3 <= 1;
46              a <= 1;
47              b <= 1;
48              c <= 1;
49              d <= 1;

```

```

48     c <= 1;
49     d <= 1;
50     e <= 1;
51     f <= 1;
52     g <= 1;
53     end
54     else if(state == 3'b001) //state 1 gives the led outputs to the AN0
55     begin
56         an0 <= 0;
57         an1 <= 1;
58         an2 <= 1;
59         an3 <= 1;
60         a <= a0;
61         b <= b0;
62         c <= c0;
63         d <= d0;
64         e <= e0;
65         f <= f0;
66         g <= g0;
67     end
68     else if(state == 3'b010) //state 2 gives the led outputs to the AN1
69     begin
70         an0 <= 1;
71         an1 <= 0;
72         an2 <= 1;
73         an3 <= 1;
74         a <= a1;
75         b <= b1;
76         c <= c1;
77         d <= d1;
78         e <= e1;
79         f <= f1;
80         g <= g1;
81     end
82     else if(state == 3'b011) //state 3 gives the led outputs to the AN2
83     begin
84         an0 <= 1;
85         an1 <= 1;
86         an2 <= 0;
87         an3 <= 1;
88         a <= a2;
89         b <= b2;
90         c <= c2;
91         d <= d2;
92         e <= e2;
93         f <= f2;
94         g <= g2;
95     end
96     else if(state == 3'b100) //state 4 gives the led outputs to the AN3
97     begin
98         an0 <= 1;
99         an1 <= 1;
100        an2 <= 1;
101        an3 <= 0;
102        a <= a3;
103        b <= b3;
104        c <= c3;
105        d <= d3;
106        e <= e3;
107        f <= f3;
108        g <= g3;
109    end
110    else //For other states default inputs and outputs
111    begin
112        an0 <= 1;
113        an1 <= 1;
114        an2 <= 1;
115        an3 <= 1;
116        a <= 1;
117        b <= 1;
118        c <= 1;
119        d <= 1;
120        e <= 1;
121        f <= 1;
122        g <= 1;
123    end
124 end
125 endmodule

```


cons.xdc:

```
1  ## This file is a general .xdc for the Basys3 rev B board
2  ## To use it in a project:
3  ## - uncomment the lines corresponding to used pins
4  ## - rename the used ports (in each line, after get_ports) according to the top level signal names in the project
5
6  ## Clock signal
7  set_property PACKAGE_PIN W5 [get_ports clk]
8  set_property IOSTANDARD LVCMOS33 [get_ports clk]
9  create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports clk]
10
11 ## Switches
12 set_property PACKAGE_PIN V17 [get_ports {passSW[0]}]
13 set_property IOSTANDARD LVCMOS33 [get_ports {passSW[0]}]
14 set_property PACKAGE_PIN V16 [get_ports {passSW[1]}]
15 set_property IOSTANDARD LVCMOS33 [get_ports {passSW[1]}]
16 set_property PACKAGE_PIN W16 [get_ports {passSW[2]}]
17 set_property IOSTANDARD LVCMOS33 [get_ports {passSW[2]}]
18 set_property PACKAGE_PIN W17 [get_ports {passSW[3]}]
19 set_property IOSTANDARD LVCMOS33 [get_ports {passSW[3]}]
20 set_property PACKAGE_PIN W15 [get_ports {passSW[4]}]
21 set_property IOSTANDARD LVCMOS33 [get_ports {passSW[4]}]
22 set_property PACKAGE_PIN V15 [get_ports {passSW[5]}]
23 set_property IOSTANDARD LVCMOS33 [get_ports {passSW[5]}]
24 set_property PACKAGE_PIN W14 [get_ports {passSW[6]}]
25 set_property IOSTANDARD LVCMOS33 [get_ports {passSW[6]}]
26 set_property PACKAGE_PIN W13 [get_ports {passSW[7]}]
27 set_property IOSTANDARD LVCMOS33 [get_ports {passSW[7]}]
28 set_property PACKAGE_PIN V2 [get_ports {passSW[8]}]
29 set_property IOSTANDARD LVCMOS33 [get_ports {passSW[8]}]
30 set_property PACKAGE_PIN T3 [get_ports {passSW[9]}]
31 set_property IOSTANDARD LVCMOS33 [get_ports {passSW[9]}]
32 set_property PACKAGE_PIN T2 [get_ports {passSW[10]}]
33 set_property IOSTANDARD LVCMOS33 [get_ports {passSW[10]}]
34 set_property PACKAGE_PIN R3 [get_ports {passSW[11]}]
35 set_property IOSTANDARD LVCMOS33 [get_ports {passSW[11]}]
36
37 set_property PACKAGE_PIN U1 [get_ports {chPASS}]
38 set_property IOSTANDARD LVCMOS33 [get_ports {chPASS}]
39 set_property PACKAGE_PIN T1 [get_ports {countersW}]
40 set_property IOSTANDARD LVCMOS33 [get_ports {countersW}]
41 set_property PACKAGE_PIN R2 [get_ports {bankSW}]
42 set_property IOSTANDARD LVCMOS33 [get_ports {bankSW}]
43
44
45 ## LEDs
46 set_property PACKAGE_PIN U16 [get_ports {t0LED}]
47 set_property IOSTANDARD LVCMOS33 [get_ports {t0LED}]
48 set_property PACKAGE_PIN E19 [get_ports {t1LED}]
49 set_property IOSTANDARD LVCMOS33 [get_ports {t1LED}]
50
51 set_property PACKAGE_PIN V14 [get_ports {wait}]
52 set_property IOSTANDARD LVCMOS33 [get_ports {wait}]
53
54 set_property PACKAGE_PIN L1 [get_ports {bnkLED}]
55 set_property IOSTANDARD LVCMOS33 [get_ports {bnkLED}]
56
57
58 ##7 segment display
59 # NOTE: abcdefg signals are connected in reverse order
60 set_property PACKAGE_PIN W7 [get_ports {g}] #normally a
61 set_property IOSTANDARD LVCMOS33 [get_ports {g}] #normally a
62 set_property PACKAGE_PIN W6 [get_ports {f}] #normally b
63 set_property IOSTANDARD LVCMOS33 [get_ports {f}] #normally b
64 set_property PACKAGE_PIN U8 [get_ports {e}] #normally c
65 set_property IOSTANDARD LVCMOS33 [get_ports {e}] #normally c
66 set_property PACKAGE_PIN V8 [get_ports {d}]
67 set_property IOSTANDARD LVCMOS33 [get_ports {d}]
68 set_property PACKAGE_PIN U5 [get_ports {c}] #normally e
69 set_property IOSTANDARD LVCMOS33 [get_ports {c}] #normally e
70 set_property PACKAGE_PIN V5 [get_ports {b}] #normally f
71 set_property IOSTANDARD LVCMOS33 [get_ports {b}] #normally f
72 set_property PACKAGE_PIN U7 [get_ports {a}] #normally g
73 set_property IOSTANDARD LVCMOS33 [get_ports {a}] #normally g
74
75
76 set_property PACKAGE_PIN U2 [get_ports {an0}]
77 set_property IOSTANDARD LVCMOS33 [get_ports {an0}]
78 set_property PACKAGE_PIN U4 [get_ports {an1}]
79 set_property IOSTANDARD LVCMOS33 [get_ports {an1}]
80 set_property PACKAGE_PIN V4 [get_ports {an2}]
81 set_property IOSTANDARD LVCMOS33 [get_ports {an2}]
82 set_property PACKAGE_PIN W4 [get_ports {an3}]
83 set_property IOSTANDARD LVCMOS33 [get_ports {an3}]
84
85
86 ##Buttons
87
88 set_property PACKAGE_PIN T18 [get_ports Cbtn]
89 set_property IOSTANDARD LVCMOS33 [get_ports Cbtn]
90 set_property PACKAGE_PIN W19 [get_ports T1btn]
91 set_property IOSTANDARD LVCMOS33 [get_ports T1btn]
92 set_property PACKAGE_PIN T17 [get_ports T0btn]
93 set_property IOSTANDARD LVCMOS33 [get_ports T0btn]
94 set_property PACKAGE_PIN U17 [get_ports rst]
95 set_property IOSTANDARD LVCMOS33 [get_ports rst]
96
```